1. What is a while loop?

A while loop allows a program to **repeat a block of code** as long as a condition is True.
In plain English:
"While this condition is true, keep running this code."
This is useful when:

- You **don't know ahead of time** how many times the code should run
- You want the program to keep running **until the user decides to stop**
- You are building **interactive programs** (like a calculator)

2. Basic while loop structure

python Copy code

```python
while condition:
    code that repeats
```

- The `condition` must be something that becomes **True or False**
- If the condition is `True`, the loop runs
- If the condition is `False`, the loop stops

Example:

python Copy code

```python
x = 1

while x <= 5:
    print(x)
    x = x + 1
```

Output:

Copy code

```
1
2
3
4
5
```

3. Why loops need a stopping condition

If the condition **never becomes False**, the loop will run forever.
This is called an **infinite loop**.

Bad example (infinite loop):

```python
x = 1

while x <= 5:
    print(x)
```

Why it's bad:

- `x` never changes
- The condition is always `True`
- The program never stops

Corrected version:

```python
x = 1

while x <= 5:
    print(x)
    x = x + 1
```

4. Using while loops with user input

while loops are often used with input() to repeat until the user chooses to stop.

Example:

```python
choice = "yes"

while choice == "yes":
    print("Hello!")
    choice = input("Type yes to continue: ")
```

This loop:

- Starts with `choice = "yes"`
- Keeps running while the user types `"yes"`
- Stops when they type something else

5. while True and breaking the loop

A common pattern is to use while True and **manually stop** the loop.

Example:

```python
while True:
    user_input = input("Type quit to stop: ")

    if user_input == "quit":
        break

    print("You typed:", user_input)
```

Explanation:

- `while True` means the loop always runs
- `break` immediately stops the loop
- This is very common in menus and calculators

6. break — stopping a loop early

The `break` keyword:

- Immediately exits the loop
- Skips all remaining code inside the loop

Example:

```python
while True:
    number = input("Enter a number or q to quit: ")

    if number == "q":
        break

    print("You entered:", number)
```

7. while loops with numbers

Example: counting down

python

Copy code

```python
count = 5

while count > 0:
    print(count)
    count = count - 1

print("Done!")
```

Output:

Copy code

```
5
4
3
2
1
Done!
```

8. Common mistakes beginners make

❌ **Forgetting to update the condition variable**

python Copy code

```python
x = 1
while x < 5:
    print(x)   # infinite loop
```

❌ **Using `==` instead of `=`**

python Copy code

```python
x == x + 1   # comparison, NOT assignment
```

Correct:

python Copy code

```python
x = x + 1
```

❌ **Expecting input() to be a number automatically**

python Copy code

```python
num = input("Enter a number: ")
# num is a string!
```

Correct:

python Copy code

```python
num = float(input("Enter a number: "))
```

9. Why while loops are perfect for the calculator assignment

A calculator needs to:

- Keep running
- Ask the user what they want
- Stop only when the user chooses to quit

This is exactly what while loops are designed for.

Example pattern:

python Copy code

```python
while True:
    choice = input("Choose an option or quit: ")

    if choice == "quit":
        break

    # do calculator work here
```

10. Key takeaways (summary)

- while loops repeat code **as long as a condition is True**
- Every while loop must have a way to **stop**
- break stops a loop immediately
- continue skips to the next loop iteration
- while loops are ideal for **interactive programs**
- Most menus and calculators rely on while True